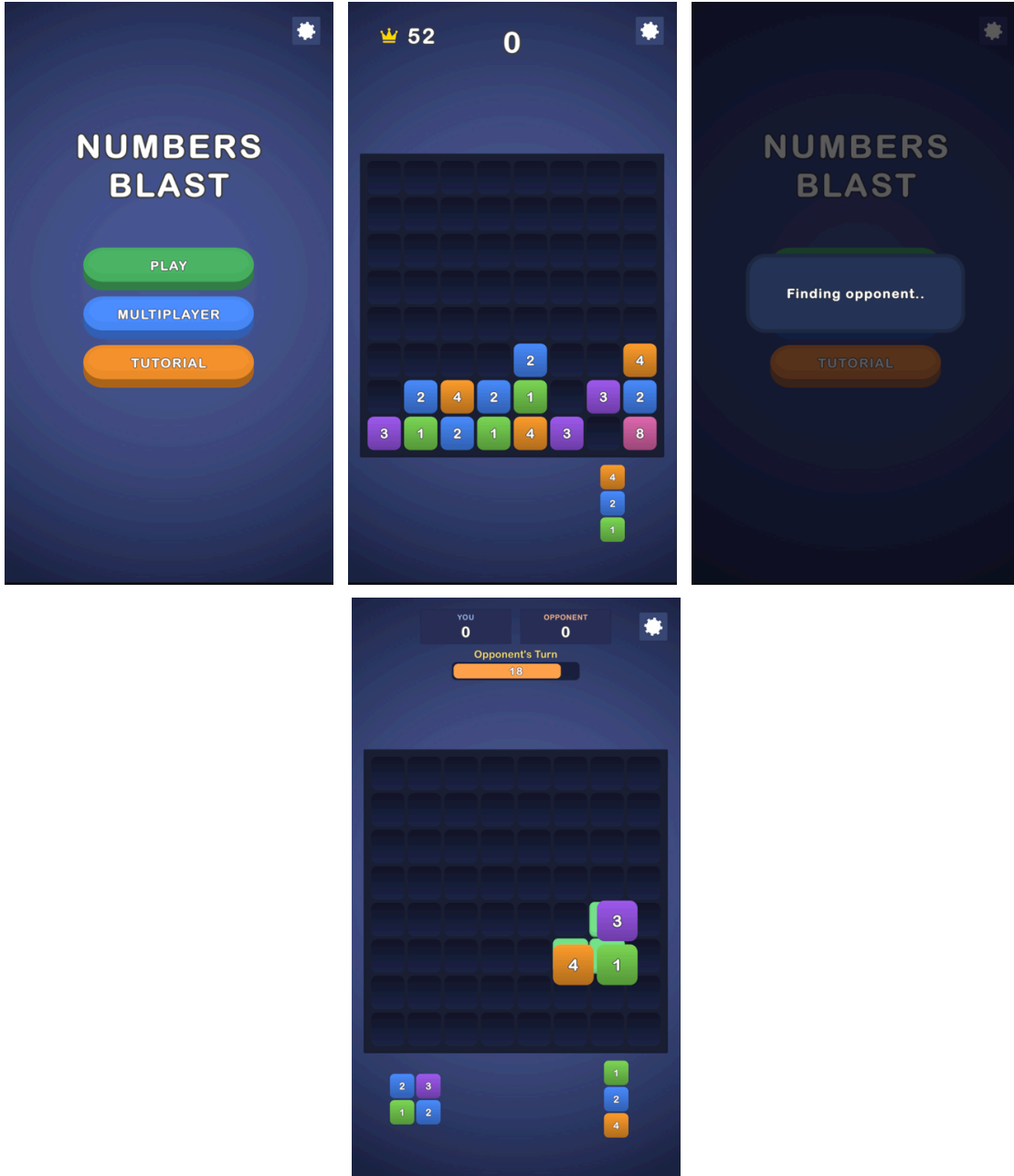


Numbers Blast

Musab Kara

Numbers Blast

Block Blast tarzı, sayı birleřtirmeli bir bulmaca prototipi: yerleřtirilen blok, eř deęerli komřularını kendine katar (zincirleme devam eder), dolan satır/sütunlar temizlenip skor verir. Çekirdek oyunun yanında, inandırıcı bir AI rakibe karřı sahte gerçek-zamanlı bir multiplayer modu ierir.



Nasıl çalıştırılır

1. Depoyu **Unity 6000.3.8f1** ile açın.
2. `Assets/Scenes/Game.unity` sahnesini açıp **Play**'e basın.
3. Testler: *Window* ▶ *General* ▶ *Test Runner* — 25 EditMode + 5 PlayMode.

Mimari

Tek assembly; klasörler namespace'lerle bire bir eşleşir.

Core	Değişmez veri + enum'lar
Data	ScriptableObject'ler (board, parça şekilleri, tutorial adımları)
Gameplay	Saf mantık, UI'sız (board, tray, hamle pipeline'ı, skor)
App	Kompozisyon kökü: <code>GameSessionController</code> + oturum yardımcıları
Presentation	Görseller, animasyonlar, ortak yerleştirme önizlemesi
Input	<code>PieceDragController</code> (fare ve dokunma tek yol)
UI	Menü, skorboard, tur/sayaç, tutorial, oyun sonu, eşleştirme
Tutorial	3 adımlı zorunlu tutorial
Opponent	Maç döngüsü, hamle değerlendirici, insansı sunum
Settings	SFX / müzik / titreşim + ayarlar paneli

Tek doğruluk kaynağı `PlacementService.ApplyMove` (yerleştir → merge → temizle): önizleme, gerçek hamle, AI değerlendirmesi ve fail-state aynı pipeline'ı kullanır — gördüğünüz önizleme ile sonuç yapısal olarak ayrılmaz. `Gameplay` katmanı hiçbir UI/Input tipini referans etmez.

Öne çıkan kararlar

- **Önce merge, sonra clear** — sıra kuralın parçasıdır ve önizleme aynısını gösterir; skor yalnız satır/sütun temizliğinden gelir (kesişim bir kez sayılır).
- **Tray içeriği modeldedir** (`TrayModel`); parça tüketimi tek noktada ve yalnızca hamle doğrulandıktan sonra yapılır.
- **Part 2:** iki turda da görünen 20 sn sayaç, ekranda görünür timeout cezası, tamamen ekran üzerinde oynayan rakip; ayarlar maçı durdurmaz.
- **AI** oyuncuyla aynı pipeline'la değerlendirir; maç içi rubber-band maçları yakın tutar, bariz en iyi hamle asla pas geçilmez, sahte gezinme asla gerçek hamleden iyi görünmez.
- DI framework'ü, service locator, event bus, kullanılmayan interface yok.

Kararların gerekçeleri ve detaylı anlatım: [README.pdf](#)

Bilinen notlar

- Tutorial yalnızca ilk açılışta çalışır; menüden tekrar oynatılabilir.

- Palet dışındaki merge değerleri kararlı bir golden-ratio tonu alır.
- Rakip turu tipik ~4–5 sn sürer; her zaman gösterdiği sayacın çok içinde biter.

Gelecek geliştirmeler

Rubber-band eşiğine bağlı zorluk seçici · rakip turu için opsiyonel süre tavanı · uçan skor yazıları için havuzlama.

Numbers Blast — Mimari ve Tasarım Kararları

Bu doküman, projedeki ana tercihlerin **neden** yapıldığını özetler. Amaç kodu yeniden anlatmak değil; her kararın arkasındaki akıl yürütmeyi ve reddedilen alternatifleri görünür kılmaktır. (Kod ve README: github.com/SkyWalker2506/Numbers-Blast)

1. Genel yapı: tek assembly, klasör = namespace

Core → Data → Gameplay → App → Presentation / Input / UI / Tutorial / Opponent / Settings

Saf oyun mantığı (`Gameplay`) hiçbir UI/Input tipini referans etmez; bu sınır sözleşmeyle değil **namespace ile** korunur (katmanda tek bir görsel/input using'i yoktur, grep ile doğrulanabilir). Her şeyi bilen tek sınıf kompozisyon köküdür (`GameSessionController`).

Neden böyle: Bu ölçekte asıl risk, mantığın UI'a sızıp test edilemez hale gelmesidir. Namespace sınırı bunu sıfır maliyetle çözer; saf katman sayesinde kuralların tamamı sahnesiz, milisaniyelik EditMode testleriyle kilitlenebildi.

Neden DI framework'ü / service locator / event bus yok: Bu ölçekte hepsi törendir. Service locator ve global event bus akışı görünmezleştirir; explicit referanslar (Initialize ile elden verilen) “kim kimi çağırıyor” sorusunu kodun kendisine cevaplatır. Projede kullanılmayan tek bir interface ya da soyutlama yoktur — “no unnecessary classes” beklentisi bilinçli bir tasarım hedefiydi.

2. Tek hamle pipeline'ı — projenin omurgası

`PlacementService.ApplyMove(board, piece, anchor)` = yerleştir → merge'leri çöz → dolu satırları temizle. Bu tek metod **dört** tüketici tarafından çağrılır: canlı sürükleme önizlemesi (scratch board'da), gerçek hamle, AI'nın aday değerlendirmesi ve fail-state kontrolü.

Neden: Kural tek yerde yaşarsa önizleme, AI ve gerçek sonuç **yapısal olarak** ayrışamaz — “gördüğün şey olacak olandır” garantisi teste değil mimariye dayanır. Alternatif (her tüketiciye kendi mantığı) üç kopya kural ve kaçınılmaz sapma demektir.

Önizleme neden scratch board’da: `BoardModel` düz bir `int[]` olduğu için `CopyFrom` ile kopyalamak ucuzdur; gerçek simülasyonu koşturmak, “tahmin eden” ikinci bir highlight mantığı yazmaktan hem daha doğru hem daha ucuzdur. (Testi de vardır: önizleme gerçek board’u asla değiştirmez.)

3. Kural kararları

- **Önce merge, sonra clear:** Sıra deterministik ve tek geçişlidir. Bir merge, dolu görünen satırı boşaltabilir — bu bir yan etki değil kuralın tanımıdır ve önizleme aynısını gösterdiği için oyuncu sürprizle karşılaşmaz. Ters sıra, zincirleme merge’lerle birlikte belirsizleşiyordu.
- **Merge skor vermez; skor yalnız satır/sütun temizliğinden gelir** ve kesişim hücresi bir kez sayılır — puanlama tek, açıklanabilir kaynaktan beslenir.
- **Parça üretimi** parça içinde komşu eş değerlere izin vermez: parça doğar doğmaz kendi kendine merge olamaz; oyuncunun gördüğü her merge kendi hamlesinin sonucudur.

4. Durum yönetimi

- Oyun akışı küçük bir state alanıyla yönetilir (Menu / PlayerTurn / Resolving / OpponentTurn / GameOver); state ataması **tek noktadan** geçer.
- Ayarlar paneli input’u state’i bozarak değil, ayrı bir kilit (`InputGate`) keser — panel herhangi bir anda açılabilir bile gerçek oyun durumu asla ezilmez.
- Tray’in içeriği görselde değil, oturuma ait `TrayModel` ’dedir; fail-state tespiti ve AI değerlendirmesi **modeli** okur. Parça tüketimi tek noktada ve yalnızca hamle doğrulandıktan sonra yapılır (model + görsel birlikte) — “yarım tüketim” sınıfı hatalar yapısal olarak kapalıdır.
- Kompozisyon kökü büyüyünce üç odaklı **düz sınıfa** bölündü (HUD düzeni, oyun sonu koreografisi, input kilidi). Düz sınıf tercihinin nedeni: MonoBehaviour olmadıkları için sahne kabloları hiç değişmedi — bölme, davranış ve sahneyi riske atmadan yapıldı.

5. Part 2: multiplayer bir illüzyondur ve illüzyon tutarlılıktan doğar

Tek büyük numara yerine küçük tutarlılıkların toplamı tercih edildi: paylaşılan board ve tray, sahte “Finding opponent...” ekranı, **iki turda da görünen** 20 saniyelik geri sayım (rakibinki kendi rengine), ekranda görünür “Time’s up! -5%” cezası ve tamamen ekran üzerinde oynayan bir rakip. Ayarlar maçı **durdurmaz** — gerçek bir online rakip beklemez; panel yalnızca yerel oyuncunun girişini kilitler.

AI kararı — iyi oynasın, kusursuz olmasın: Her aday, oyuncuyla aynı pipeline’la puanlanır (ayrı bir “AI kuralı” yoktur); en iyiler arasından ağırlıklı-rastgele seçim yapılır. Üzerine üç inandırıcılık katmanı eklendi:

1. **Maç içi rubber-band:** seçim genişliği anlık skor farkını izler — önde gevşer, geride en iyi hamlesini oynar. Profil/kalıcılık gerektirmez; tek maçta bile hissedilir.
2. **Bariz hamle kuralı:** en iyi aday açık farkla öndeyse (ör. satır temizliği) asla pas geçilmez — “temizliği görüp saçma yere koyan” robot görüntüsü yapısal olarak engellenir.
3. **Dürüstlük kuralları:** rakip, gerçekte oynayacağından daha iyi görünen bir hücrenin üzerinde asla sahte gezinmez; tereddüdün süresi ve deneme sayısı zara değil, kararın gerçekten ne kadar yakın olduğuna bağlıdır. Bariz hamlede dosdoğru gider, yakın kararda düşünür — kararlılık da tereddüt kadar insansıdır.

Sunum, karardan ayırdır: saf ve seed’lenebilir bir plancı hamleyi “beat” listesine çevirir (düşünme, deneme, nadir yanlış bırakma, yanlış taşı alıp vazgeçme), görsel katman yalnızca oynatır. Bu ayırım sayesinde koreografi mantığı unit-test edilebilir.

6. Test stratejisi

İki katman, iki amaç: **25 EditMode** testi kural katmanını kilitler (merge zinciri, skor, kesişim, önizleme dokunulmazlığı, ceza matematiği, fail-state kenarları, spawn kuralı, plancı davranışları) — sahnesiz ve CI-dostu. **5 PlayMode** testi gerçek sahneyi sürer (solo akış, Vs AI başlangıcı, timeout’un turu devretmesi, ayar kilidinin temizlenmesi). Geliştirme sırasında bulunan her gerçek hata birer regression testine dönüştü — kapsam, hata geçmişinin haritasıdır.

7. Performans (Android hedefi)

Sıcak yollarda LINQ yok; önizleme yalnız hedef hücre değişince yeniden hesaplanır (sabit sürüklemelerde sıfır GC); sayaç metni saniyede en fazla bir kez string üretir; temizlik parlamaları havuzludur; AI, aday başına yeni board ayırmak yerine tek scratch board’u yeniden kullanır; **Update** yalnızca tur sayacı ve safe-area’dadır. Canvas **Expand** modunda ölçeklenir: 1080×1920 tasarım kutusu her ekran oranına tamamen sığar.

8. Bilinçli ertelenenler

Zorluk seçici (rubber-band eşiği zaten Inspector’da), rakip turu için süre tavanı, uçan skor yazılarının havuzlanması. Hiçbiri unutulmadı; bu ölçekte karşılığı olmayan karmaşıklık olarak sınıflandırıldı ve README’de açıkça listelendi.